

RetroSpectrum Data Storage and SQLCipher Reference

August 13, 2026

Abstract

This document summarizes how RetroSpectrum stores operational data in SQL databases and how those databases are protected at rest. It is based on the supplied `DatabaseCrypto.c` and `DataStore.c` sources. The active design uses a 32-byte master key, domain-separated database keys, SQLCipher 4.6.1 or newer, transactional SQL storage, and SHA-512 integrity checks for generic stored documents. Classification data is stored as structured SQL rows rather than as a persisted CSV file.

1 Storage Pipeline

The local data path is:

```
Workstation data → DATASTORE save/load/list/delete API → Open retrospectrum.data.db
through DatabaseCrypto → Derive 256-bit database key from the master key → Apply key
to SQLCipher and verify the database can be read → Prepared SQL statements +
transaction → stored.documents BLOBs or structured classification rows →
SQLCipher-protected database file on disk
```

When RetroSpectrum is authenticated to a remote server, the public DataStore API routes save, load, list, and delete operations through `SecureNetwork`; the server-side DataStore functions then perform the SQL operation on the server's encrypted database.

2 Master Key and Database Key Derivation

2.1 Master key

RetroSpectrum uses a 32-byte master key. If the default key does not yet exist and no database already exists, it is generated with `RAND_priv_bytes()` and created as an owner-only regular file with mode `0600`. Existing key files must also be regular, owner-controlled, exactly 32 bytes, and restricted to owner access.

Relevant code (DatabaseCrypto.c lines 583–601):

```
583 if (RAND_priv_bytes(master, DATABASE_CRYPTO_MASTER_BYTES) != 1) {
584     ...
589 old_mask = umask(0077);
590 descriptor = open(path, O_WRONLY | O_CREAT | O_EXCL |
591                   O_CLOEXEC | O_NOFOLLOW, 0600);
600 database_crypto_write_all(descriptor, master,
601                           DATABASE_CRYPTO_MASTER_BYTES);
```

While cached in memory, the master key is protected with `mlock()` where available, marked `MADV_DONTDUMP` where supported, and wiped with `OPENSSL_cleanse()` during replacement and process cleanup.

2.2 Domain-separated database keys

The same master key is not passed directly to both databases. RetroSpectrum computes HMAC-SHA-512 over a database-specific domain string and uses the first 32 bytes of the digest:

$$K_{db} = \text{Trunc}_{32}(\text{HMAC-SHA512}(K_{master}, D_{db})). \quad (1)$$

The domains are:

Database	Derivation domain
Authentication	RetroSpectrum SQLCipher auth database key v1
Operational data	RetroSpectrum SQLCipher operational database key v1

This produces separate 256-bit keys for `auth.db` and `retrospectrum_data.db` even though both originate from the same master key.

3 SQLCipher Protection

RetroSpectrum refuses plaintext SQLite databases and requires SQLCipher 4.6.1 or newer. The derived 32-byte key is supplied to SQLCipher as a hexadecimal raw key. The connection is configured with a 4096-byte cipher page size, HMAC-SHA-512 page authentication, PBKDF2-HMAC-SHA-512 as the configured SQLCipher KDF algorithm, and SQLCipher memory security.

Relevant code (DatabaseCrypto.c lines 975–992):

```

975 if (!database || !key || !database_crypto_has_sqlcipher(database)) {
976     "RetroSpectrum requires SQLCipher 4.6.1 or newer; ...";
977 }
978 database_crypto_hex(key, key_hex);
979 snprintf(sql, sizeof(sql), "PRAGMA key = \"x'%s'\";", key_hex);
980 PRAGMA cipher_page_size = 4096;
981 PRAGMA cipher_hmac_algorithm = HMAC_SHA512;
982 PRAGMA cipher_kdf_algorithm = PBKDF2_HMAC_SHA512;
983 PRAGMA cipher_memory_security = ON;

```

After applying the key, RetroSpectrum queries `sqlite_master`. Failure is treated as an incorrect key or damaged database. It also explicitly checks for the normal SQLite format 3 plaintext header and refuses such databases.

Note: the supplied source explicitly configures SQLCipher and its HMAC/KDF settings, but does not explicitly select the underlying SQLCipher page-encryption cipher by name. This document therefore does not assign one beyond what the source states.

4 SQL Data Model

4.1 Generic documents

Most workstation data is stored in `stored_documents` using a composite key of document kind and document name. The row contains the case number, content BLOB, a 64-byte SHA-512 digest, and creation/update timestamps. Supported document kinds include case management, correlation, bitstream classifier data, case images, and case colors.

On save, the content digest is calculated before the transaction. The data and digest are written using prepared statements and bound parameters. On load, RetroSpectrum recalculates SHA-512 and compares it with the stored value using `CRYPTO_memcmp()` before returning the content.

$$H = \text{SHA512}(\text{content}), \quad \text{accept load only if } H = H_{\text{stored}}. \quad (2)$$

This application-level digest is in addition to SQLCipher's configured page authentication.

4.2 Classification records

Classification data uses structured SQL tables rather than one persisted CSV/BLOB document. `classification_sets` stores document-level metadata, while `classification_records` stores fields such as case number, signal name, frequency, bandwidth, times, modulation, signal class, country, coordinates, notes, and file name. Records are ordered by `row_order` and linked to their set by a foreign key with `ON DELETE CASCADE`.

The workstation-facing compatibility representation is parsed into SQL fields when saving and reconstructed in memory from those rows when loading. The compatibility text is not the persistent storage format.

5 Transactions and Deletion

DataStore configures foreign keys, `journal_mode=DELETE`, `synchronous=FULL`, `secure_delete=ON`, and `temp_store=MEMORY`. Save and delete operations use `BEGIN IMMEDIATE`; successful operations `COMMIT`, while failures `ROLLBACK`.

For classification deletion, removing the parent row from `classification_sets` also removes its child `classification_records` through the foreign-key cascade. Generic documents are deleted by their document kind and document name.

6 Compact Security Summary

Layer	Implementation
Master key	32 random bytes from <code>RAND_priv_bytes</code> ; owner-only key file.
Database key derivation	HMAC-SHA-512 with database-specific domain separation; first 32 bytes used.
Database engine	SQLCipher 4.6.1 or newer; plaintext SQLite refused.
SQLCipher integrity/KDF config	HMAC-SHA-512; PBKDF2-HMAC-SHA-512 configured; 4096-byte cipher pages.
Generic content integrity	SHA-512 digest stored with each BLOB and verified on load.
SQL safety	Prepared statements and bound values for document and classification data.
Consistency	Immediate transactions with commit/rollback; foreign-key enforcement.
Sensitive memory	Master/derived key material is cleansed; cached master is locked/-dntdump where supported.